In [4]:
```
import pandas as pd

file_path = r"D:\QP_project\Data File.csv"

df = pd.read_csv(file_path)

print(df.head())
print(df.columns)
print(df.shape)
```

```
       Date        BIL        BND        IEF        IWM        LQD        QQQ  \
0  2008/1/2  73.685081  44.447693  56.861046  58.768089  53.036777  43.411434
1  2008/1/3  73.636826  44.505161  56.977837  58.119095  52.861790  43.592278
2  2008/1/4  73.620773  44.556881  57.127064  56.367619  53.296749  41.680492
3  2008/1/7  73.685081  44.568340  57.230923  56.492737  53.231750  41.482422
4  2008/1/8  73.636826  44.614319  57.354206  54.749081  53.131786  40.405964

         SHY         SPY        TIP  ...  XLC        XLE        XLF  \
0  62.264160  103.778244  64.034721  ...  NaN  22.710733  16.290852
1  62.362442  103.728149  64.250885  ...  NaN  22.979259  16.187456
2  62.453213  101.186127  64.232880  ...  NaN  22.139393  15.727901
3  62.415386  101.100220  64.220848  ...  NaN  22.053690  15.768114
4  62.506104   99.467590  64.268890  ...  NaN  21.668043  15.193683

         XLI        XLK        XLP  XLRE        XLU        XLV        XLY
0  27.078758  10.242213  17.477942   NaN  11.182665  25.555614  12.790724
1  27.099852  10.257907  17.385593   NaN  11.166725  25.709204  12.647725
2  26.544373   9.857941  17.311718   NaN  11.251743  25.445902  12.250493
3  26.213875   9.771672  17.514877   NaN  11.485541  25.950579  12.294186
4  25.630255   9.512869  17.385593   NaN  11.488203  26.162685  12.055848

[5 rows x 25 columns]
Index(['Date', 'BIL', 'BND', 'IEF', 'IWM', 'LQD', 'QQQ', 'SHY', 'SPY', 'TIP',
       'TLT', 'VGIT', 'VGLT', 'VGSH', 'XLB', 'XLC', 'XLE', 'XLF', 'XLI', 'XLK',
       'XLP', 'XLRE', 'XLU', 'XLV', 'XLY'],
      dtype='object')
(4529, 25)
```

In [5]:
```
##Task 1: SPY Buy-and-Hold
```

In [8]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv(
    r"D:\QP_project\Data File.csv",
    parse_dates=["Date"]
)

df.columns = df.columns.str.replace("\\", "", regex=False).str.strip()
df = df.sort_values("Date").reset_index(drop=True)

use_cols = ["Date", "SPY", "BIL"]
data = df[use_cols].copy().dropna().reset_index(drop=True)

data["spy_ret"] = data["SPY"].pct_change()
data["rf_ret"] = data["BIL"].pct_change()
```

```
bt = data[(data["Date"] >= "2019-01-01") & (data["Date"] <= "2023-12-31")].copy(
bt = bt.dropna().reset_index(drop=True)

bt["nav"] = (1 + bt["spy_ret"]).cumprod()
bt["cum_return"] = bt["nav"] - 1
bt["peak"] = bt["nav"].cummax()
bt["drawdown"] = bt["nav"] / bt["peak"] - 1

ann_factor = 252
days = len(bt)

ann_return = bt["nav"].iloc[-1] ** (ann_factor / days) - 1
ann_vol = bt["spy_ret"].std(ddof=1) * np.sqrt(ann_factor)

excess_ret = bt["spy_ret"] - bt["rf_ret"]
rf_ann = (1 + bt["rf_ret"].mean()) ** ann_factor - 1
sharpe = excess_ret.mean() / bt["spy_ret"].std(ddof=1) * np.sqrt(ann_factor)

downside = bt.loc[bt["spy_ret"] < 0, "spy_ret"]
downside_dev = downside.std(ddof=1) * np.sqrt(ann_factor)
sortino = (ann_return - rf_ann) / downside_dev if downside_dev != 0 else np.nan

mdd = bt["drawdown"].min()

var_95 = bt["spy_ret"].quantile(0.05)
cvar_95 = bt.loc[bt["spy_ret"] <= var_95, "spy_ret"].mean()

market_excess = bt["spy_ret"] - bt["rf_ret"]
portfolio_excess = bt["spy_ret"] - bt["rf_ret"]

beta = portfolio_excess.cov(market_excess) / market_excess.var(ddof=1)
alpha_daily = portfolio_excess.mean() - beta * market_excess.mean()
alpha_ann = alpha_daily * ann_factor

tracking_error = (bt["spy_ret"] - bt["spy_ret"]).std(ddof=1) * np.sqrt(ann_facto
information_ratio = np.nan

rolling_1y = bt.set_index("Date")["nav"].pct_change(252)
rolling_1y_monthly = rolling_1y.resample("ME").last().dropna()

summary = pd.DataFrame({
    "Metric": [
        "Backtest Start",
        "Backtest End",
        "Trading Days",
        "Cumulative Return",
        "Annualized Return",
        "Annualized Volatility",
        "Risk-Free Annual Return",
        "Sharpe Ratio",
        "Sortino Ratio",
        "Maximum Drawdown",
        "Daily VaR 95%",
        "Daily CVaR 95%",
        "Beta vs SPY",
        "Jensen Alpha Annualized",
        "Tracking Error vs SPY",
        "Information Ratio vs SPY"
    ],
    "Value": [
```

```
        bt["Date"].min().strftime("%Y-%m-%d"),
        bt["Date"].max().strftime("%Y-%m-%d"),
        len(bt),
        bt["cum_return"].iloc[-1],
        ann_return,
        ann_vol,
        rf_ann,
        sharpe,
        sortino,
        mdd,
        var_95,
        cvar_95,
        beta,
        alpha_ann,
        tracking_error,
        information_ratio
    ]
})

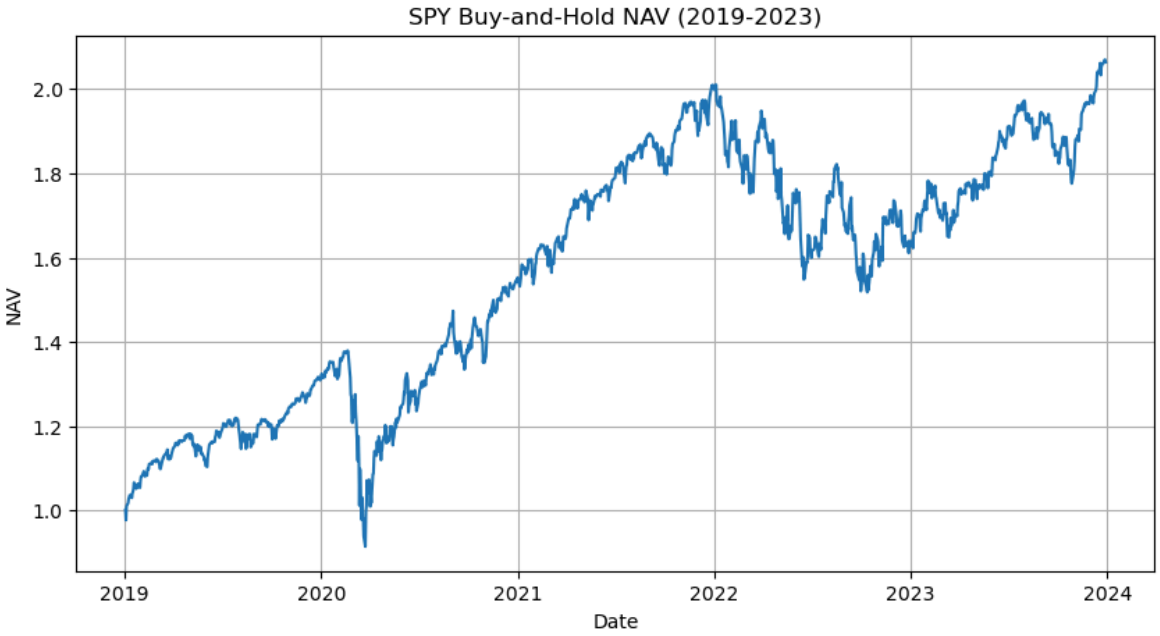
print(summary)

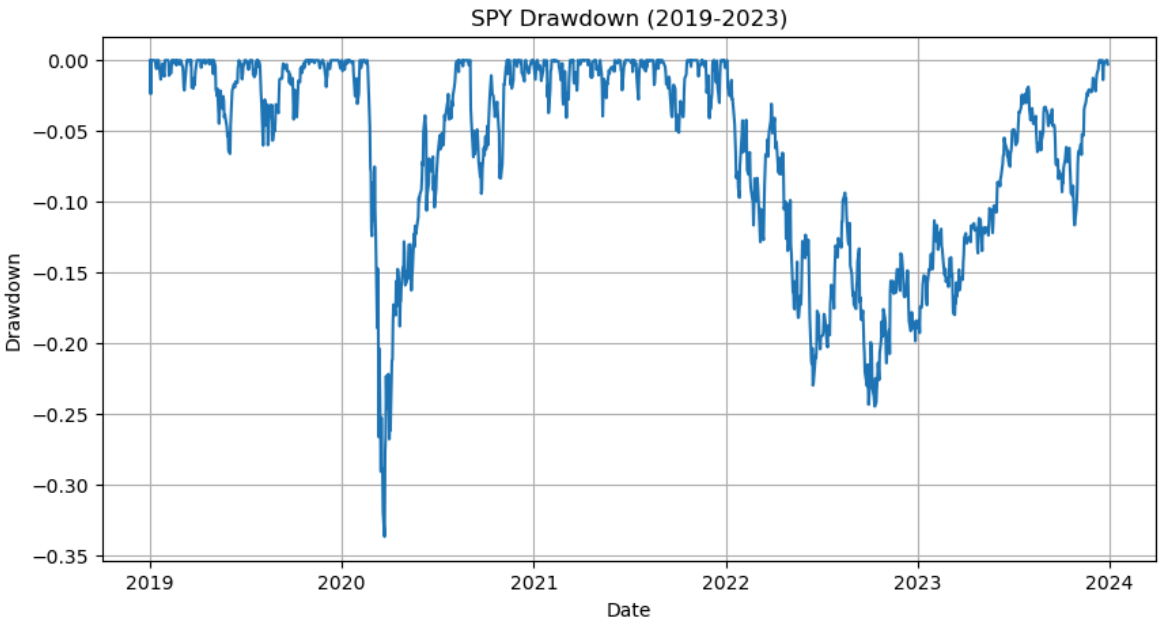
plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["nav"])
plt.title("SPY Buy-and-Hold NAV (2019-2023)")
plt.xlabel("Date")
plt.ylabel("NAV")
plt.grid(True)
plt.show()

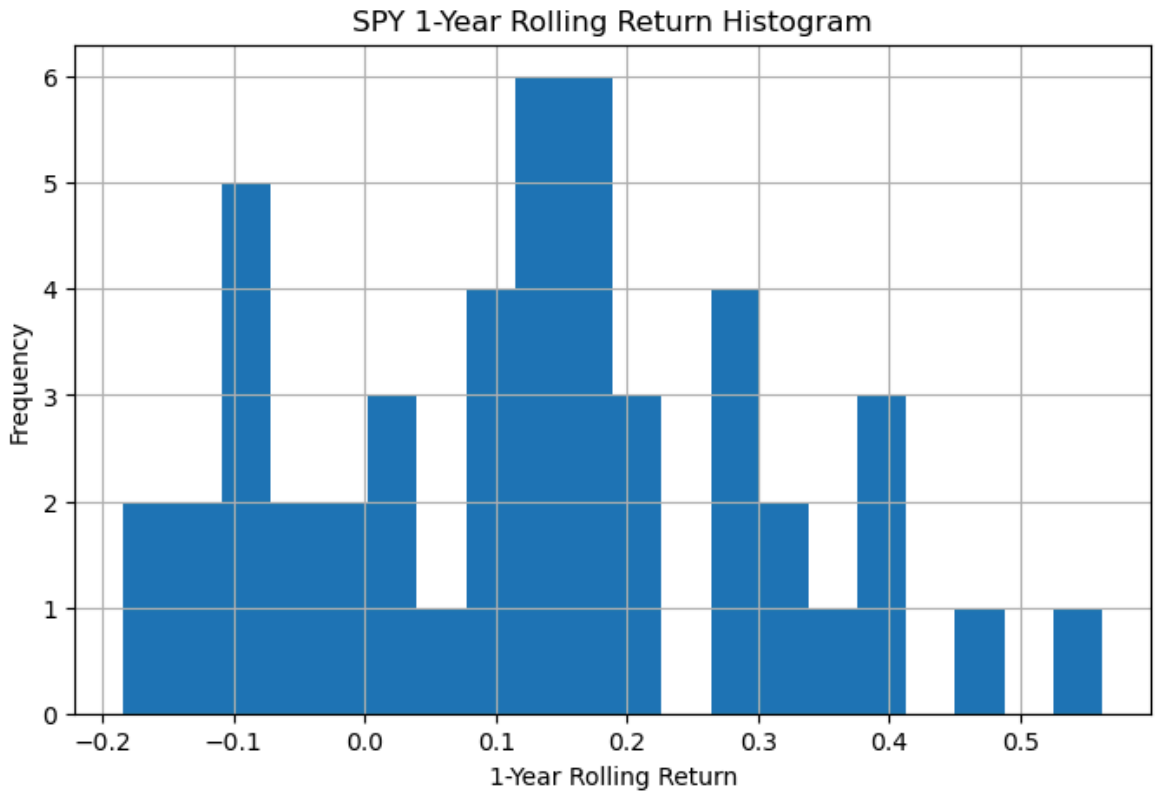
plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["drawdown"])
plt.title("SPY Drawdown (2019-2023)")
plt.xlabel("Date")
plt.ylabel("Drawdown")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.hist(rolling_1y_monthly.values, bins=20)
plt.title("SPY 1-Year Rolling Return Histogram")
plt.xlabel("1-Year Rolling Return")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```

	Metric	Value
0	Backtest Start	2019-01-02
1	Backtest End	2023-12-29
2	Trading Days	1258
3	Cumulative Return	1.063713
4	Annualized Return	0.156192
5	Annualized Volatility	0.209874
6	Risk-Free Annual Return	0.017241
7	Sharpe Ratio	0.715689
8	Sortino Ratio	0.804529
9	Maximum Drawdown	-0.337173
10	Daily VaR 95%	-0.019278
11	Daily CVaR 95%	-0.032151
12	Beta vs SPY	1.0
13	Jensen Alpha Annualized	0.0
14	Tracking Error vs SPY	0.0
15	Information Ratio vs SPY	NaN







In [11]:
```
##Task 2: 60/40 SPY/BND(rebalance quarterly)
```

In [10]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv(
    r"D:\QP_project\Data File.csv",
    parse_dates=["Date"]
)

df.columns = df.columns.str.replace("\\", "", regex=False).str.strip()
df = df.sort_values("Date").reset_index(drop=True)

use_cols = ["Date", "SPY", "BND", "BIL"]
data = df[use_cols].copy().dropna().reset_index(drop=True)

data["spy_ret"] = data["SPY"].pct_change()
data["bnd_ret"] = data["BND"].pct_change()
data["rf_ret"] = data["BIL"].pct_change()

bt = data[(data["Date"] >= "2019-01-01") & (data["Date"] <= "2023-12-31")].copy(
bt = bt.dropna().reset_index(drop=True)

bt["month"] = bt["Date"].dt.month
bt["year"] = bt["Date"].dt.year
bt["quarter"] = bt["Date"].dt.to_period("Q")

quarter_start_dates = bt.groupby("quarter")["Date"].min().tolist()
rebalance_dates = set(quarter_start_dates)

w_spy = 0.6
w_bnd = 0.4
weights_spy = []
weights_bnd = []
```

```
port_rets = []

for i in range(len(bt)):
    if i == 0:
        weights_spy.append(w_spy)
        weights_bnd.append(w_bnd)
        port_rets.append(w_spy * bt.loc[i, "spy_ret"] + w_bnd * bt.loc[i, "bnd_r
        continue

    current_date = bt.loc[i, "Date"]

    if current_date in rebalance_dates:
        w_spy = 0.6
        w_bnd = 0.4

    port_ret = w_spy * bt.loc[i, "spy_ret"] + w_bnd * bt.loc[i, "bnd_ret"]
    port_rets.append(port_ret)

    spy_gross = w_spy * (1 + bt.loc[i, "spy_ret"])
    bnd_gross = w_bnd * (1 + bt.loc[i, "bnd_ret"])
    total_gross = spy_gross + bnd_gross

    w_spy = spy_gross / total_gross
    w_bnd = bnd_gross / total_gross

    weights_spy.append(w_spy)
    weights_bnd.append(w_bnd)

bt["w_spy"] = weights_spy
bt["w_bnd"] = weights_bnd
bt["port_ret"] = port_rets

bt["nav"] = (1 + bt["port_ret"]).cumprod()
bt["cum_return"] = bt["nav"] - 1
bt["peak"] = bt["nav"].cummax()
bt["drawdown"] = bt["nav"] / bt["peak"] - 1

ann_factor = 252
days = len(bt)

ann_return = bt["nav"].iloc[-1] ** (ann_factor / days) - 1
ann_vol = bt["port_ret"].std(ddof=1) * np.sqrt(ann_factor)

excess_ret = bt["port_ret"] - bt["rf_ret"]
rf_ann = (1 + bt["rf_ret"].mean()) ** ann_factor - 1
sharpe = excess_ret.mean() / bt["port_ret"].std(ddof=1) * np.sqrt(ann_factor)

downside = bt.loc[bt["port_ret"] < 0, "port_ret"]
downside_dev = downside.std(ddof=1) * np.sqrt(ann_factor)
sortino = (ann_return - rf_ann) / downside_dev if downside_dev != 0 else np.nan

mdd = bt["drawdown"].min()

var_95 = bt["port_ret"].quantile(0.05)
cvar_95 = bt.loc[bt["port_ret"] <= var_95, "port_ret"].mean()

market_excess = bt["spy_ret"] - bt["rf_ret"]
portfolio_excess = bt["port_ret"] - bt["rf_ret"]

beta = portfolio_excess.cov(market_excess) / market_excess.var(ddof=1)
```

```
alpha_daily = portfolio_excess.mean() - beta * market_excess.mean()
alpha_ann = alpha_daily * ann_factor

active_ret = bt["port_ret"] - bt["spy_ret"]
tracking_error = active_ret.std(ddof=1) * np.sqrt(ann_factor)
information_ratio = (active_ret.mean() / active_ret.std(ddof=1) * np.sqrt(ann_fa

rolling_1y = bt.set_index("Date")["nav"].pct_change(252)
rolling_1y_monthly = rolling_1y.resample("ME").last().dropna()

summary = pd.DataFrame({
    "Metric": [
        "Backtest Start",
        "Backtest End",
        "Trading Days",
        "Cumulative Return",
        "Annualized Return",
        "Annualized Volatility",
        "Risk-Free Annual Return",
        "Sharpe Ratio",
        "Sortino Ratio",
        "Maximum Drawdown",
        "Daily VaR 95%",
        "Daily CVaR 95%",
        "Beta vs SPY",
        "Jensen Alpha Annualized",
        "Tracking Error vs SPY",
        "Information Ratio vs SPY"
    ],
    "Value": [
        bt["Date"].min().strftime("%Y-%m-%d"),
        bt["Date"].max().strftime("%Y-%m-%d"),
        len(bt),
        bt["cum_return"].iloc[-1],
        ann_return,
        ann_vol,
        rf_ann,
        sharpe,
        sortino,
        mdd,
        var_95,
        cvar_95,
        beta,
        alpha_ann,
        tracking_error,
        information_ratio
    ]
})

print(summary)

rebalance_table = bt[bt["Date"].isin(quarter_start_dates)][["Date"]].copy()
rebalance_table["Target_SPY_Weight"] = 0.6
rebalance_table["Target_BND_Weight"] = 0.4

print("\nQuarterly Rebalance Dates:")
print(rebalance_table)

plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["nav"])
```

```
plt.title("60/40 SPY-BND Portfolio NAV (2019-2023)")
plt.xlabel("Date")
plt.ylabel("NAV")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["drawdown"])
plt.title("60/40 SPY-BND Drawdown (2019-2023)")
plt.xlabel("Date")
plt.ylabel("Drawdown")
plt.grid(True)
plt.show()

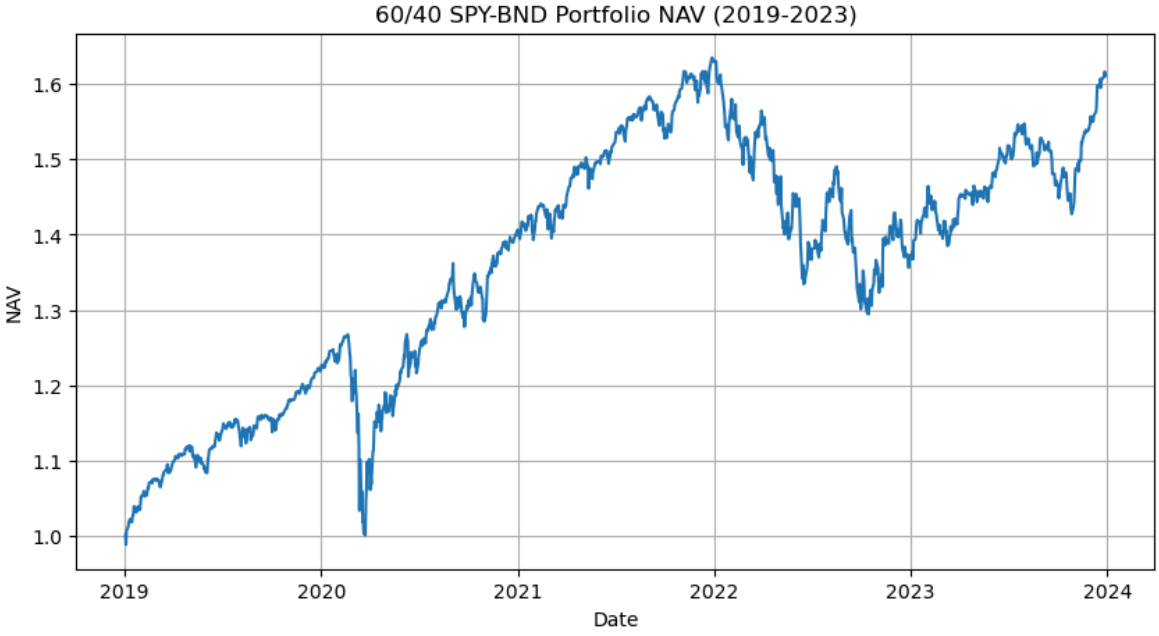
plt.figure(figsize=(8, 5))
plt.hist(rolling_1y_monthly.values, bins=20)
plt.title("60/40 SPY-BND 1-Year Rolling Return Histogram")
plt.xlabel("1-Year Rolling Return")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

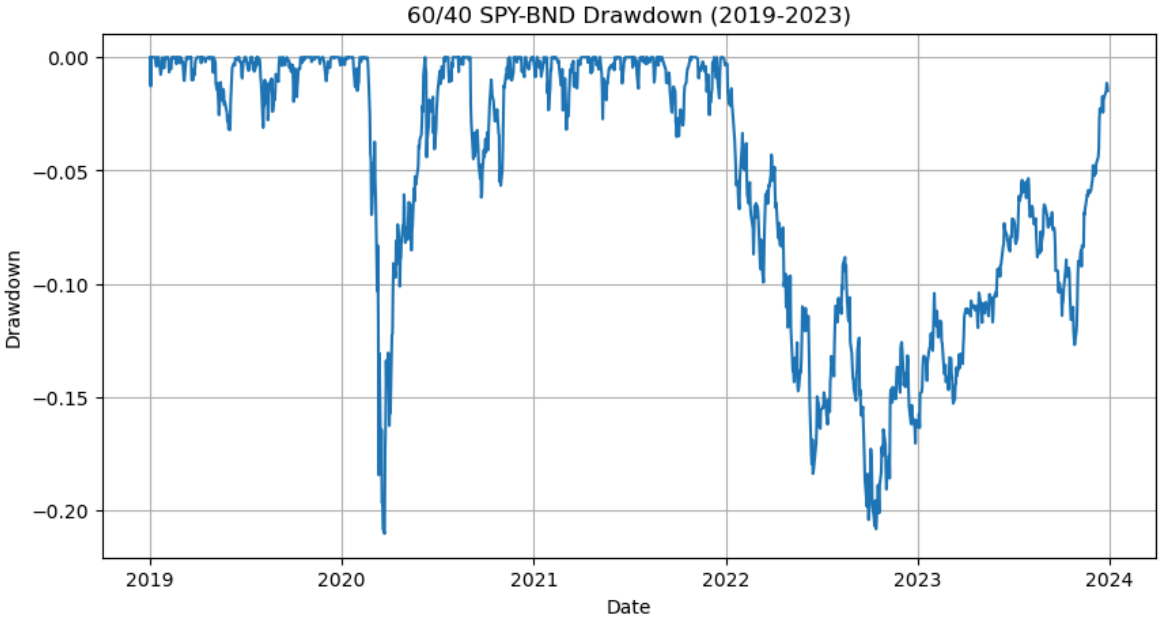
plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["w_spy"], label="SPY Weight")
plt.plot(bt["Date"], bt["w_bnd"], label="BND Weight")
plt.title("60/40 Portfolio Weight Drift Between Rebalances")
plt.xlabel("Date")
plt.ylabel("Weight")
plt.legend()
plt.grid(True)
plt.show()
```

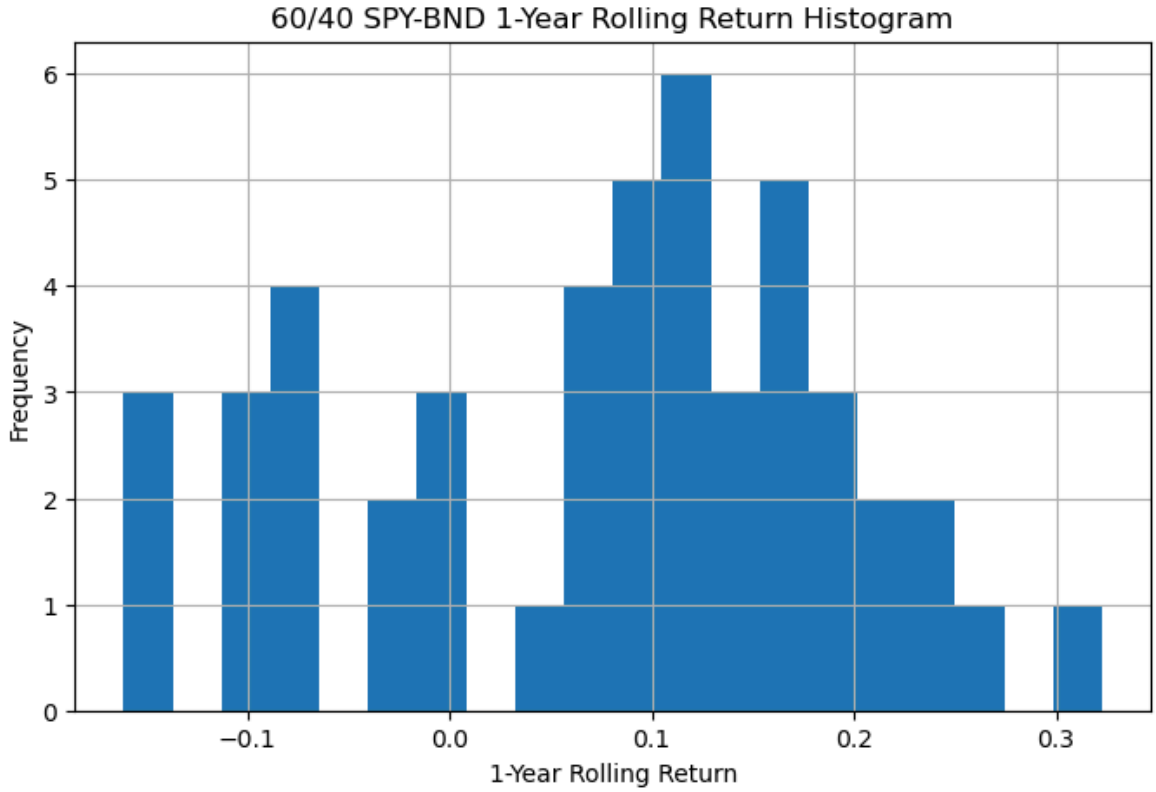
	Metric	Value
0	Backtest Start	2019-01-02
1	Backtest End	2023-12-29
2	Trading Days	1258
3	Cumulative Return	0.610874
4	Annualized Return	0.100217
5	Annualized Volatility	0.131491
6	Risk-Free Annual Return	0.017241
7	Sharpe Ratio	0.662419
8	Sortino Ratio	0.75949
9	Maximum Drawdown	-0.210334
10	Daily VaR 95%	-0.011666
11	Daily CVaR 95%	-0.019913
12	Beta vs SPY	0.612086
13	Jensen Alpha Annualized	-0.004836
14	Tracking Error vs SPY	0.086113
15	Information Ratio vs SPY	-0.732789

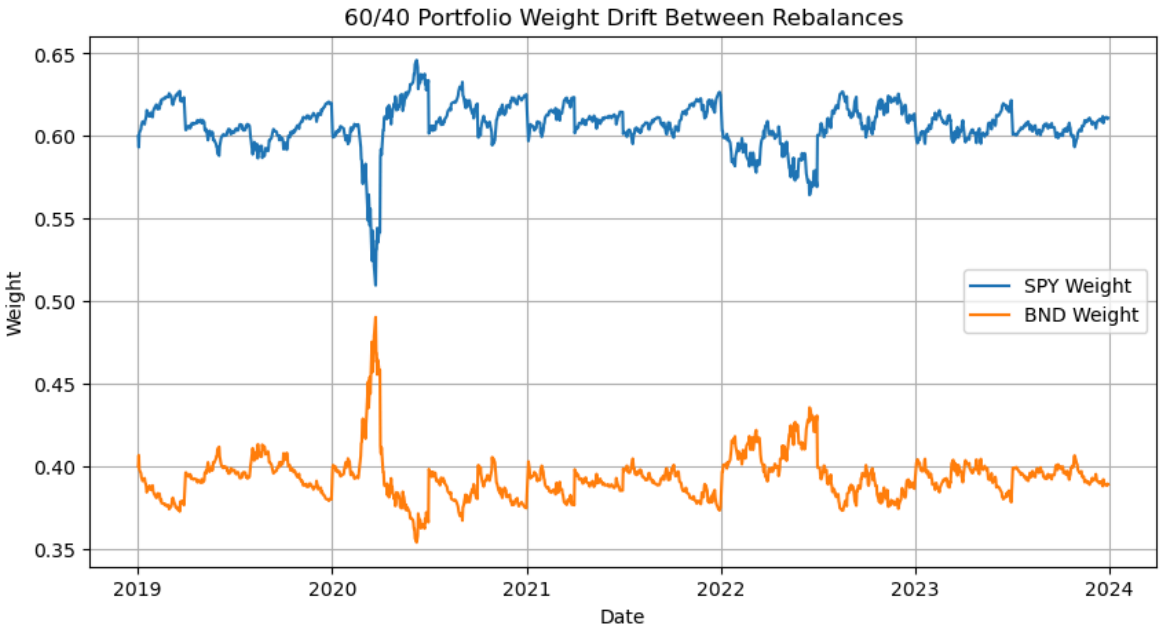
Quarterly Rebalance Dates:

	Date	Target_SPY_Weight	Target_BND_Weight
0	2019-01-02	0.6	0.4
61	2019-04-01	0.6	0.4
124	2019-07-01	0.6	0.4
188	2019-10-01	0.6	0.4
252	2020-01-02	0.6	0.4
314	2020-04-01	0.6	0.4
377	2020-07-01	0.6	0.4
441	2020-10-01	0.6	0.4
505	2021-01-04	0.6	0.4
566	2021-04-01	0.6	0.4
629	2021-07-01	0.6	0.4
693	2021-10-01	0.6	0.4
757	2022-01-03	0.6	0.4
819	2022-04-01	0.6	0.4
881	2022-07-01	0.6	0.4
945	2022-10-03	0.6	0.4
1008	2023-01-03	0.6	0.4
1070	2023-04-03	0.6	0.4
1132	2023-07-03	0.6	0.4
1195	2023-10-02	0.6	0.4




60/40 SPY-BND Drawdown (2019-2023)
Drawdown
0.00
−0.05
−0.10
−0.15
−0.20
2019
2020
2021
2022
2023
2024
Date


60/40 SPY-BND 1-Year Rolling Return Histogram
Frequency
0
1
2
3
4
5
6
−0.1
0.0
0.1
0.2
0.3
1-Year Rolling Return



In [12]:
```
##Task 3: Markowitz / Fast Algorithm
```

In [13]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

df = pd.read_csv(
    r"D:\QP_project\Data File.csv",
    parse_dates=["Date"]
)

df.columns = df.columns.str.replace("\\", "", regex=False).str.strip()
df = df.sort_values("Date").reset_index(drop=True)

sector_cols = ["XLC", "XLY", "XLP", "XLE", "XLF", "XLV", "XLI", "XLB", "XLRE", "
use_cols = ["Date", "SPY", "BIL"] + sector_cols

data = df[use_cols].copy()
data = data.sort_values("Date").reset_index(drop=True)

for col in ["SPY", "BIL"] + sector_cols:
    data[col + "_ret"] = data[col].pct_change()

ret_cols = [c + "_ret" for c in sector_cols]

data = data[(data["Date"] >= "2018-01-01") & (data["Date"] <= "2023-12-31")].cop
data = data.reset_index(drop=True)

bt = data[data["Date"] >= "2019-01-01"].copy().reset_index(drop=True)
bt["quarter"] = bt["Date"].dt.to_period("Q")
rebalance_dates = bt.groupby("quarter")["Date"].min().tolist()

def max_sharpe_weights(mu, cov, rf):
    n = len(mu)

    def neg_sharpe(w):
        port_ret = np.dot(w, mu)
        port_vol = np.sqrt(np.dot(w, np.dot(cov, w)))
        if port_vol <= 0:
```

```
            return 1e6
        return -(port_ret - rf) / port_vol

    x0 = np.ones(n) / n
    bounds = [(0.0, 1.0)] * n
    constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1.0}]

    res = minimize(
        neg_sharpe,
        x0,
        method="SLSQP",
        bounds=bounds,
        constraints=constraints
    )

    if not res.success:
        return x0
    return res.x

weights_records = []
portfolio_returns = []

current_weights = np.ones(len(sector_cols)) / len(sector_cols)

for i in range(len(bt)):
    current_date = bt.loc[i, "Date"]

    if current_date in rebalance_dates:
        est = data[data["Date"] < current_date].copy()
        est = est.tail(252)

        est = est.dropna(subset=ret_cols + ["BIL_ret", "SPY_ret"]).reset_index(d

        if len(est) < 200:
            current_weights = np.ones(len(sector_cols)) / len(sector_cols)
        else:
            mu = est[ret_cols].mean().values
            cov = est[ret_cols].cov().values
            rf = est["BIL_ret"].mean()

            cov = cov + np.eye(len(sector_cols)) * 1e-6

            current_weights = max_sharpe_weights(mu, cov, rf)

        record = {"Date": current_date}
        for j, col in enumerate(sector_cols):
            record[col] = current_weights[j]
        weights_records.append(record)

    day_ret = bt.loc[i, ret_cols].values.astype(float)

    if np.isnan(day_ret).any():
        valid = ~np.isnan(day_ret)
        if valid.sum() == 0:
            port_ret = 0.0
        else:
            w = current_weights[valid]
            w = w / w.sum()
            port_ret = np.dot(w, day_ret[valid])
    else:
```

```
        port_ret = np.dot(current_weights, day_ret)

    portfolio_returns.append(port_ret)

bt["port_ret"] = portfolio_returns
bt["rf_ret"] = bt["BIL_ret"]
bt["spy_ret"] = bt["SPY_ret"]

bt = bt.dropna(subset=["port_ret", "rf_ret", "spy_ret"]).reset_index(drop=True)

bt["nav"] = (1 + bt["port_ret"]).cumprod()
bt["cum_return"] = bt["nav"] - 1
bt["peak"] = bt["nav"].cummax()
bt["drawdown"] = bt["nav"] / bt["peak"] - 1

ann_factor = 252
days = len(bt)

ann_return = bt["nav"].iloc[-1] ** (ann_factor / days) - 1
ann_vol = bt["port_ret"].std(ddof=1) * np.sqrt(ann_factor)

excess_ret = bt["port_ret"] - bt["rf_ret"]
rf_ann = (1 + bt["rf_ret"].mean()) ** ann_factor - 1
sharpe = excess_ret.mean() / bt["port_ret"].std(ddof=1) * np.sqrt(ann_factor)

downside = bt.loc[bt["port_ret"] < 0, "port_ret"]
downside_dev = downside.std(ddof=1) * np.sqrt(ann_factor)
sortino = (ann_return - rf_ann) / downside_dev if downside_dev != 0 else np.nan

mdd = bt["drawdown"].min()

var_95 = bt["port_ret"].quantile(0.05)
cvar_95 = bt.loc[bt["port_ret"] <= var_95, "port_ret"].mean()

market_excess = bt["spy_ret"] - bt["rf_ret"]
portfolio_excess = bt["port_ret"] - bt["rf_ret"]

beta = portfolio_excess.cov(market_excess) / market_excess.var(ddof=1)
alpha_daily = portfolio_excess.mean() - beta * market_excess.mean()
alpha_ann = alpha_daily * ann_factor

active_ret = bt["port_ret"] - bt["spy_ret"]
tracking_error = active_ret.std(ddof=1) * np.sqrt(ann_factor)
information_ratio = (active_ret.mean() / active_ret.std(ddof=1) * np.sqrt(ann_fa

rolling_1y = bt.set_index("Date")["nav"].pct_change(252)
rolling_1y_monthly = rolling_1y.resample("ME").last().dropna()

summary = pd.DataFrame({
    "Metric": [
        "Backtest Start",
        "Backtest End",
        "Trading Days",
        "Cumulative Return",
        "Annualized Return",
        "Annualized Volatility",
        "Risk-Free Annual Return",
        "Sharpe Ratio",
        "Sortino Ratio",
        "Maximum Drawdown",
```

```
        "Daily VaR 95%",
        "Daily CVaR 95%",
        "Beta vs SPY",
        "Jensen Alpha Annualized",
        "Tracking Error vs SPY",
        "Information Ratio vs SPY"
    ],
    "Value": [
        bt["Date"].min().strftime("%Y-%m-%d"),
        bt["Date"].max().strftime("%Y-%m-%d"),
        len(bt),
        bt["cum_return"].iloc[-1],
        ann_return,
        ann_vol,
        rf_ann,
        sharpe,
        sortino,
        mdd,
        var_95,
        cvar_95,
        beta,
        alpha_ann,
        tracking_error,
        information_ratio
    ]
})

weights_df = pd.DataFrame(weights_records)

print(summary)
print("\nRebalance Weights:")
print(weights_df)

plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["nav"])
plt.title("Markowitz-Sharpe Sector Portfolio NAV (2019-2023)")
plt.xlabel("Date")
plt.ylabel("NAV")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["drawdown"])
plt.title("Markowitz-Sharpe Sector Portfolio Drawdown (2019-2023)")
plt.xlabel("Date")
plt.ylabel("Drawdown")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.hist(rolling_1y_monthly.values, bins=20)
plt.title("Markowitz-Sharpe 1-Year Rolling Return Histogram")
plt.xlabel("1-Year Rolling Return")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

plt.figure(figsize=(12, 6))
for col in sector_cols:
    plt.plot(weights_df["Date"], weights_df[col], label=col)
```

```
plt.title("Quarterly Rebalance Weights")
plt.xlabel("Date")
plt.ylabel("Weight")
plt.legend(ncol=3)
plt.grid(True)
plt.show()
```

	Metric	Value
0	Backtest Start	2019-01-02
1	Backtest End	2023-12-29
2	Trading Days	1258
3	Cumulative Return	1.228456
4	Annualized Return	0.174117
5	Annualized Volatility	0.240857
6	Risk-Free Annual Return	0.017241
7	Sharpe Ratio	0.71645
8	Sortino Ratio	0.821679
9	Maximum Drawdown	-0.345626
10	Daily VaR 95%	-0.022182
11	Daily CVaR 95%	-0.035489
12	Beta vs SPY	0.945322
13	Jensen Alpha Annualized	0.03057
14	Tracking Error vs SPY	0.137169
15	Information Ratio vs SPY	0.162991

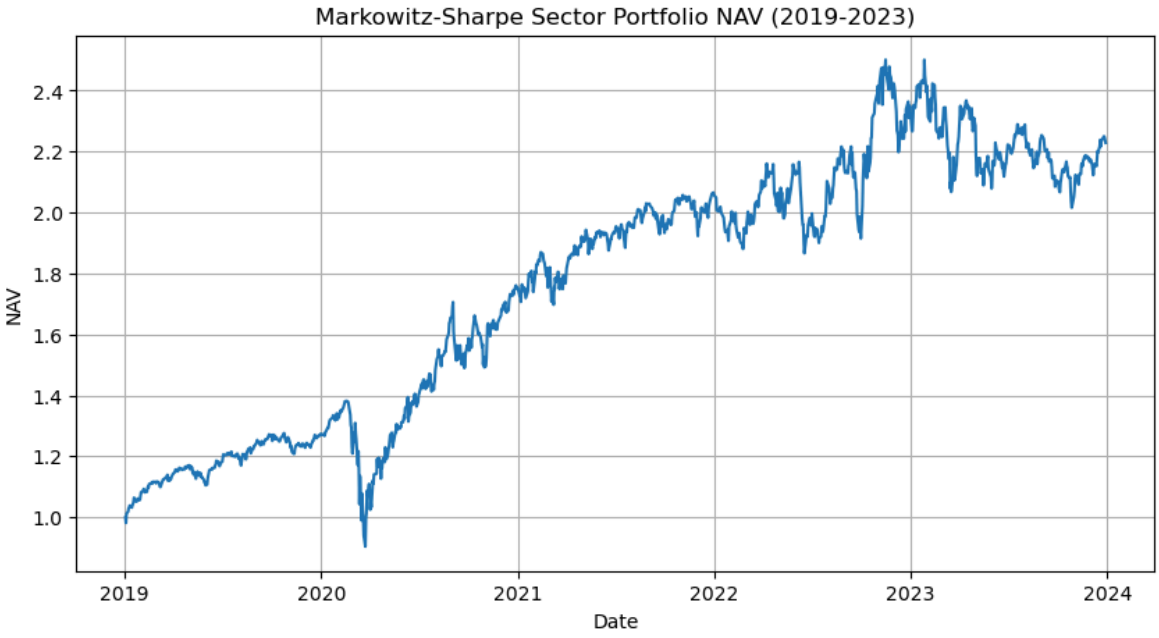
Rebalance Weights:

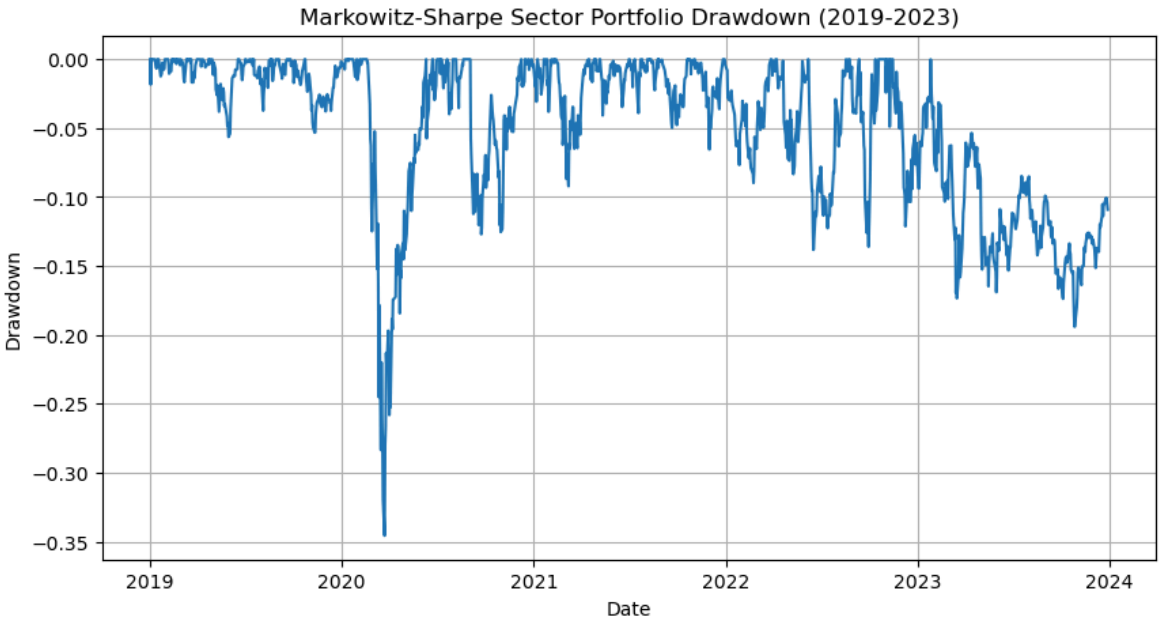
	Date	XLC	XLY	XLP	XLE
0	2019-01-02	9.090909e-02	9.090909e-02	9.090909e-02	9.090909e-02
1	2019-04-01	9.090909e-02	9.090909e-02	9.090909e-02	9.090909e-02
2	2019-07-01	2.824410e-17	9.726419e-17	2.244109e-01	0.000000e+00
3	2019-10-01	5.095522e-17	7.565931e-19	1.189857e-17	0.000000e+00
4	2020-01-02	0.000000e+00	0.000000e+00	7.019720e-02	0.000000e+00
5	2020-04-01	6.511833e-17	0.000000e+00	1.795431e-16	0.000000e+00
6	2020-07-01	0.000000e+00	3.165519e-17	3.040699e-16	6.097721e-17
7	2020-10-01	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
8	2021-01-04	0.000000e+00	0.000000e+00	0.000000e+00	1.210386e-16
9	2021-04-01	4.133175e-01	2.328774e-01	2.385245e-18	0.000000e+00
10	2021-07-01	4.359611e-01	0.000000e+00	3.303428e-18	0.000000e+00
11	2021-10-01	2.673574e-01	0.000000e+00	5.605325e-17	5.602828e-02
12	2022-01-03	1.781358e-17	1.007103e-18	0.000000e+00	1.544666e-01
13	2022-04-01	2.280467e-17	0.000000e+00	4.744007e-17	3.111063e-01
14	2022-07-01	1.216107e-16	0.000000e+00	9.183996e-18	4.476056e-01
15	2022-10-03	1.365005e-17	0.000000e+00	0.000000e+00	1.000000e+00
16	2023-01-03	0.000000e+00	7.880777e-17	0.000000e+00	1.000000e+00
17	2023-04-03	7.054835e-18	9.246293e-18	0.000000e+00	1.000000e+00
18	2023-07-03	0.000000e+00	0.000000e+00	0.000000e+00	1.054625e-16
19	2023-10-02	4.596443e-01	0.000000e+00	6.965999e-17	3.553487e-01

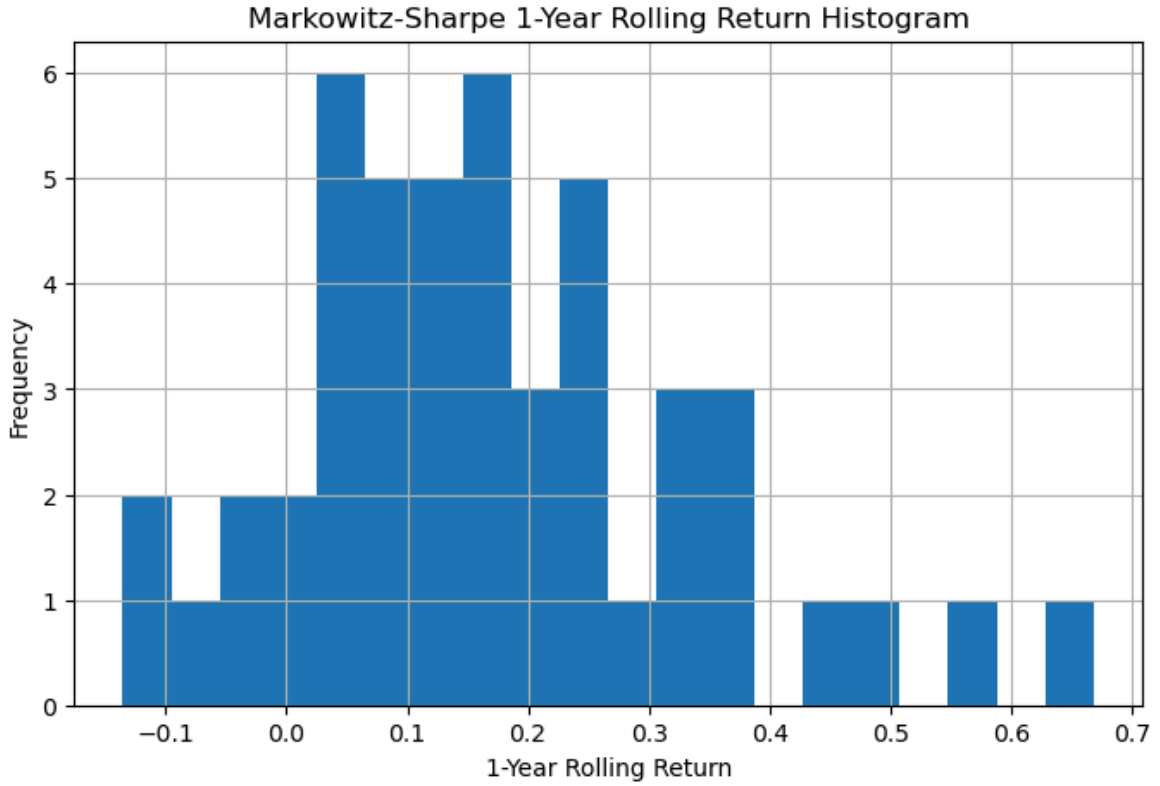
	XLF	XLV	XLI	XLB	XLRE
0	9.090909e-02	9.090909e-02	9.090909e-02	9.090909e-02	9.090909e-02
1	9.090909e-02	9.090909e-02	9.090909e-02	9.090909e-02	9.090909e-02
2	3.213326e-17	4.431100e-18	0.000000e+00	1.181211e-17	1.081091e-01
3	0.000000e+00	5.759104e-17	1.037398e-17	1.198993e-17	2.405591e-01
4	9.544158e-02	0.000000e+00	5.185982e-18	0.000000e+00	1.721720e-01
5	0.000000e+00	1.271540e-17	0.000000e+00	0.000000e+00	0.000000e+00
6	8.212190e-17	0.000000e+00	8.696505e-17	0.000000e+00	9.204401e-18
7	7.525455e-17	1.747734e-16	3.871388e-17	0.000000e+00	5.736943e-17
8	3.939855e-17	0.000000e+00	3.876058e-17	0.000000e+00	3.653494e-17
9	0.000000e+00	1.366095e-17	8.174884e-17	3.538051e-01	0.000000e+00
10	3.441617e-01	2.907511e-02	1.673518e-01	2.516704e-17	2.345034e-02
11	3.471407e-01	7.529314e-02	6.559423e-18	0.000000e+00	2.541805e-01
12	6.835425e-18	2.916117e-01	0.000000e+00	8.159106e-18	5.539217e-01
13	0.000000e+00	3.684061e-01	7.378531e-17	0.000000e+00	7.730386e-02
14	5.642297e-18	0.000000e+00	0.000000e+00	0.000000e+00	1.840413e-16
15	0.000000e+00	3.910812e-17	1.396604e-17	2.608082e-17	0.000000e+00
16	6.192908e-18	3.499951e-17	1.697638e-17	2.921692e-17	0.000000e+00
17	0.000000e+00	0.000000e+00	9.957705e-17	0.000000e+00	0.000000e+00

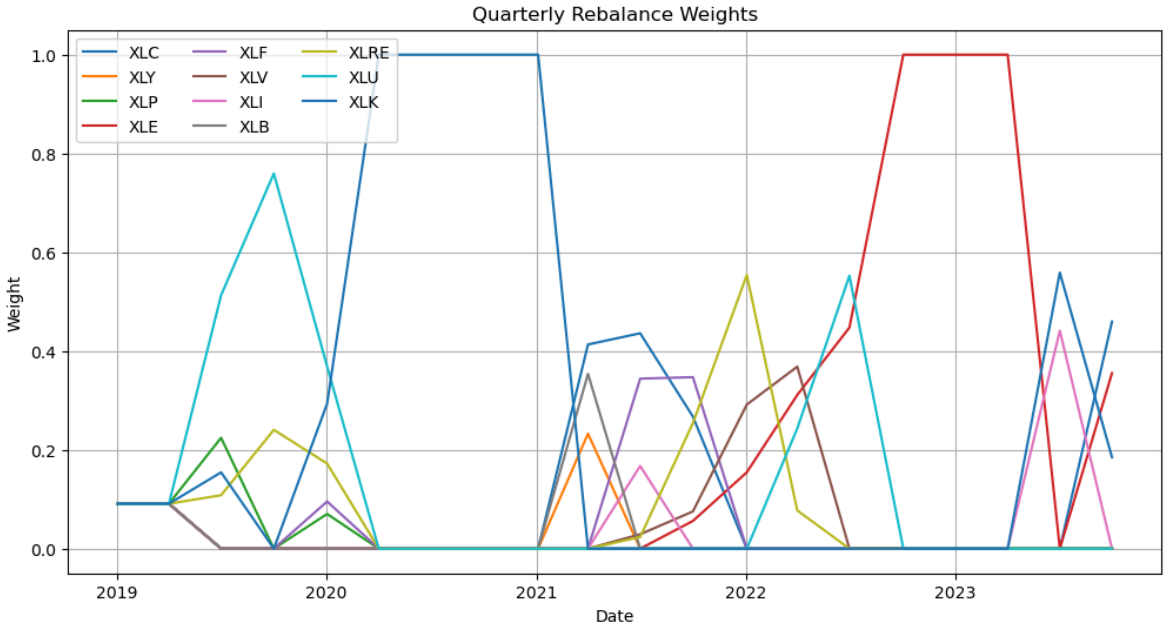
18	2.229069e-17	0.000000e+00	4.412248e-01	0.000000e+00	4.362566e-17
19	1.141665e-16	1.219727e-16	0.000000e+00	0.000000e+00	0.000000e+00

	XLU	XLK
0	9.090909e-02	9.090909e-02
1	9.090909e-02	9.090909e-02
2	5.129927e-01	1.544872e-01
3	7.594409e-01	3.774880e-17
4	3.685109e-01	2.936784e-01
5	5.082740e-16	1.000000e+00
6	0.000000e+00	1.000000e+00
7	6.742095e-19	1.000000e+00
8	4.287459e-17	1.000000e+00
9	8.673617e-19	0.000000e+00
10	1.897354e-18	0.000000e+00
11	2.629190e-17	0.000000e+00
12	2.223836e-17	0.000000e+00
13	2.431837e-01	0.000000e+00
14	5.523944e-01	0.000000e+00
15	2.359224e-16	0.000000e+00
16	2.393573e-17	1.709029e-17
17	0.000000e+00	6.764320e-17
18	1.527520e-17	5.587752e-01
19	0.000000e+00	1.850070e-01




Markowitz-Sharpe Sector Portfolio Drawdown (2019-2023)
Drawdown
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
−0.30
−0.35
2019
2020
2021
2022
2023
2024
Date


Markowitz-Sharpe 1-Year Rolling Return Histogram
Frequency
0
1
2
3
4
5
6
−0.1
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
1-Year Rolling Return



In [ ]:
```
##fast algorithm single-index
```

In [14]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv(
    r"D:\QP_project\Data File.csv",
    parse_dates=["Date"]
)

df.columns = df.columns.str.replace("\\", "", regex=False).str.strip()
df = df.sort_values("Date").reset_index(drop=True)

sector_cols = ["XLC", "XLY", "XLP", "XLE", "XLF", "XLV", "XLI", "XLB", "XLRE", "
use_cols = ["Date", "SPY", "BIL"] + sector_cols

data = df[use_cols].copy().sort_values("Date").reset_index(drop=True)

for col in ["SPY", "BIL"] + sector_cols:
    data[col + "_ret"] = data[col].pct_change()

data = data[(data["Date"] >= "2018-01-01") & (data["Date"] <= "2023-12-31")].cop

ret_cols = [c + "_ret" for c in sector_cols]

def single_index_fast_weights(est):
    market_excess = est["SPY_ret"] - est["BIL_ret"]
    sigma_m2 = market_excess.var(ddof=1)

    rows = []

    for col in sector_cols:
        asset_excess = est[col + "_ret"] - est["BIL_ret"]

        valid = pd.concat([asset_excess, market_excess], axis=1).dropna()
        if len(valid) < 60:
            rows.append([col, np.nan, np.nan, np.nan, np.nan, np.nan])
            continue
```

```
        y = valid.iloc[:, 0].values
        x = valid.iloc[:, 1].values

        beta = np.cov(y, x, ddof=1)[0, 1] / np.var(x, ddof=1)
        alpha = y.mean() - beta * x.mean()
        resid = y - (alpha + beta * x)
        resid_var = np.var(resid, ddof=1)
        mean_excess = y.mean()
        ratio = mean_excess / beta if beta > 1e-12 else np.nan

        rows.append([col, alpha, beta, resid_var, mean_excess, ratio])

    info = pd.DataFrame(
        rows,
        columns=["asset", "alpha", "beta", "resid_var", "mean_excess", "ratio"]
    )

    info = info.replace([np.inf, -np.inf], np.nan)
    info = info.dropna().copy()
    info = info[(info["beta"] > 1e-12) & (info["resid_var"] > 1e-12)].copy()

    if len(info) == 0:
        return pd.Series(np.ones(len(sector_cols)) / len(sector_cols), index=sec

    info = info.sort_values("ratio", ascending=False).reset_index(drop=True)

    A_cum = 0.0
    B_cum = 0.0
    c_list = []

    for i in range(len(info)):
        A_i = info.loc[i, "mean_excess"] * info.loc[i, "beta"] / info.loc[i, "re
        B_i = info.loc[i, "beta"] ** 2 / info.loc[i, "resid_var"]

        A_cum += A_i
        B_cum += B_i

        C_i = sigma_m2 * A_cum / (1.0 + sigma_m2 * B_cum)
        c_list.append(C_i)

    info["C"] = c_list
    info["eligible"] = info["ratio"] > info["C"]

    eligible_idx = np.where(info["eligible"].values)[0]

    if len(eligible_idx) == 0:
        return pd.Series(np.ones(len(sector_cols)) / len(sector_cols), index=sec

    k_star = eligible_idx[-1]
    C_star = info.loc[k_star, "C"]

    active = info.iloc[:k_star + 1].copy()
    active["z"] = active["beta"] / active["resid_var"] * (active["ratio"] - C_st
    active = active[active["z"] > 0].copy()

    if len(active) == 0:
        return pd.Series(np.ones(len(sector_cols)) / len(sector_cols), index=sec

    active["w"] = active["z"] / active["z"].sum()
```

```
    w = pd.Series(0.0, index=sector_cols)
    for _, row in active.iterrows():
        w[row["asset"]] = row["w"]

    if w.sum() <= 0:
        w[:] = 1.0 / len(sector_cols)
    else:
        w = w / w.sum()

    return w

bt = data[data["Date"] >= "2019-01-01"].copy().reset_index(drop=True)
bt["quarter"] = bt["Date"].dt.to_period("Q")
rebalance_dates = bt.groupby("quarter")["Date"].min().tolist()

weights_records = []
portfolio_returns = []

current_weights = pd.Series(np.ones(len(sector_cols)) / len(sector_cols), index=

for i in range(len(bt)):
    current_date = bt.loc[i, "Date"]

    if current_date in rebalance_dates:
        est = data[data["Date"] < current_date].copy().tail(252).reset_index(drc

        req_cols = ["SPY_ret", "BIL_ret"] + ret_cols
        est = est.dropna(subset=req_cols).reset_index(drop=True)

        if len(est) < 200:
            current_weights = pd.Series(np.ones(len(sector_cols)) / len(sector_c
        else:
            current_weights = single_index_fast_weights(est)

        rec = {"Date": current_date}
        for c in sector_cols:
            rec[c] = current_weights[c]
        weights_records.append(rec)

    day_ret = bt.loc[i, ret_cols].copy()
    day_ret.index = sector_cols

    valid = day_ret.notna()
    if valid.sum() == 0:
        port_ret = 0.0
    else:
        w = current_weights[valid]
        w = w / w.sum()
        port_ret = np.dot(w.values, day_ret[valid].astype(float).values)

    portfolio_returns.append(port_ret)

bt["port_ret"] = portfolio_returns
bt["rf_ret"] = bt["BIL_ret"]
bt["spy_ret"] = bt["SPY_ret"]

bt = bt.dropna(subset=["port_ret", "rf_ret", "spy_ret"]).reset_index(drop=True)

bt["nav"] = (1 + bt["port_ret"]).cumprod()
bt["cum_return"] = bt["nav"] - 1
```

```
bt["peak"] = bt["nav"].cummax()
bt["drawdown"] = bt["nav"] / bt["peak"] - 1

ann_factor = 252
days = len(bt)

ann_return = bt["nav"].iloc[-1] ** (ann_factor / days) - 1
ann_vol = bt["port_ret"].std(ddof=1) * np.sqrt(ann_factor)

excess_ret = bt["port_ret"] - bt["rf_ret"]
rf_ann = (1 + bt["rf_ret"].mean()) ** ann_factor - 1
sharpe = excess_ret.mean() / bt["port_ret"].std(ddof=1) * np.sqrt(ann_factor)

downside = bt.loc[bt["port_ret"] < 0, "port_ret"]
downside_dev = downside.std(ddof=1) * np.sqrt(ann_factor)
sortino = (ann_return - rf_ann) / downside_dev if downside_dev != 0 else np.nan

mdd = bt["drawdown"].min()

var_95 = bt["port_ret"].quantile(0.05)
cvar_95 = bt.loc[bt["port_ret"] <= var_95, "port_ret"].mean()

market_excess = bt["spy_ret"] - bt["rf_ret"]
portfolio_excess = bt["port_ret"] - bt["rf_ret"]

beta = portfolio_excess.cov(market_excess) / market_excess.var(ddof=1)
alpha_daily = portfolio_excess.mean() - beta * market_excess.mean()
alpha_ann = alpha_daily * ann_factor

active_ret = bt["port_ret"] - bt["spy_ret"]
tracking_error = active_ret.std(ddof=1) * np.sqrt(ann_factor)
information_ratio = active_ret.mean() / active_ret.std(ddof=1) * np.sqrt(ann_fac

rolling_1y = bt.set_index("Date")["nav"].pct_change(252)
rolling_1y_monthly = rolling_1y.resample("ME").last().dropna()

summary = pd.DataFrame({
    "Metric": [
        "Backtest Start",
        "Backtest End",
        "Trading Days",
        "Cumulative Return",
        "Annualized Return",
        "Annualized Volatility",
        "Risk-Free Annual Return",
        "Sharpe Ratio",
        "Sortino Ratio",
        "Maximum Drawdown",
        "Daily VaR 95%",
        "Daily CVaR 95%",
        "Beta vs SPY",
        "Jensen Alpha Annualized",
        "Tracking Error vs SPY",
        "Information Ratio vs SPY"
    ],
    "Value": [
        bt["Date"].min().strftime("%Y-%m-%d"),
        bt["Date"].max().strftime("%Y-%m-%d"),
        len(bt),
        bt["cum_return"].iloc[-1],
```

```
        ann_return,
        ann_vol,
        rf_ann,
        sharpe,
        sortino,
        mdd,
        var_95,
        cvar_95,
        beta,
        alpha_ann,
        tracking_error,
        information_ratio
    ]
})

weights_df = pd.DataFrame(weights_records)

print(summary)
print("\nRebalance Weights:")
print(weights_df)

plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["nav"])
plt.title("Fast Algorithm Single-Index Portfolio NAV (2019-2023)")
plt.xlabel("Date")
plt.ylabel("NAV")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["drawdown"])
plt.title("Fast Algorithm Single-Index Portfolio Drawdown (2019-2023)")
plt.xlabel("Date")
plt.ylabel("Drawdown")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.hist(rolling_1y_monthly.values, bins=20)
plt.title("Fast Algorithm 1-Year Rolling Return Histogram")
plt.xlabel("1-Year Rolling Return")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

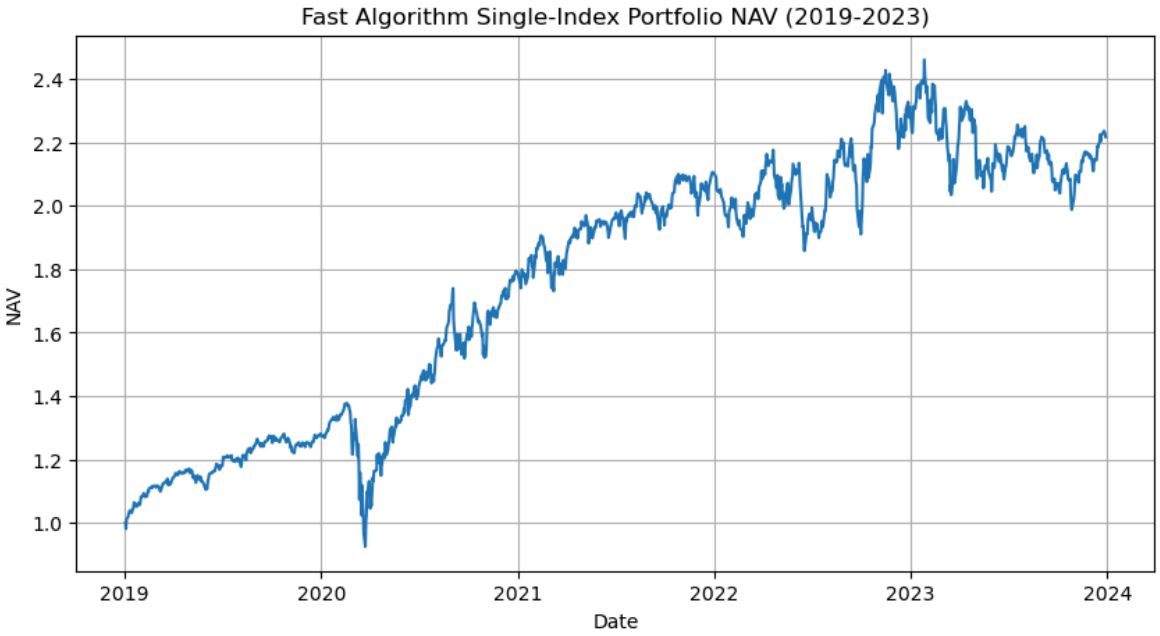
plt.figure(figsize=(12, 6))
for col in sector_cols:
    plt.plot(weights_df["Date"], weights_df[col], label=col)
plt.title("Quarterly Rebalance Weights")
plt.xlabel("Date")
plt.ylabel("Weight")
plt.legend(ncol=3)
plt.grid(True)
plt.show()
```

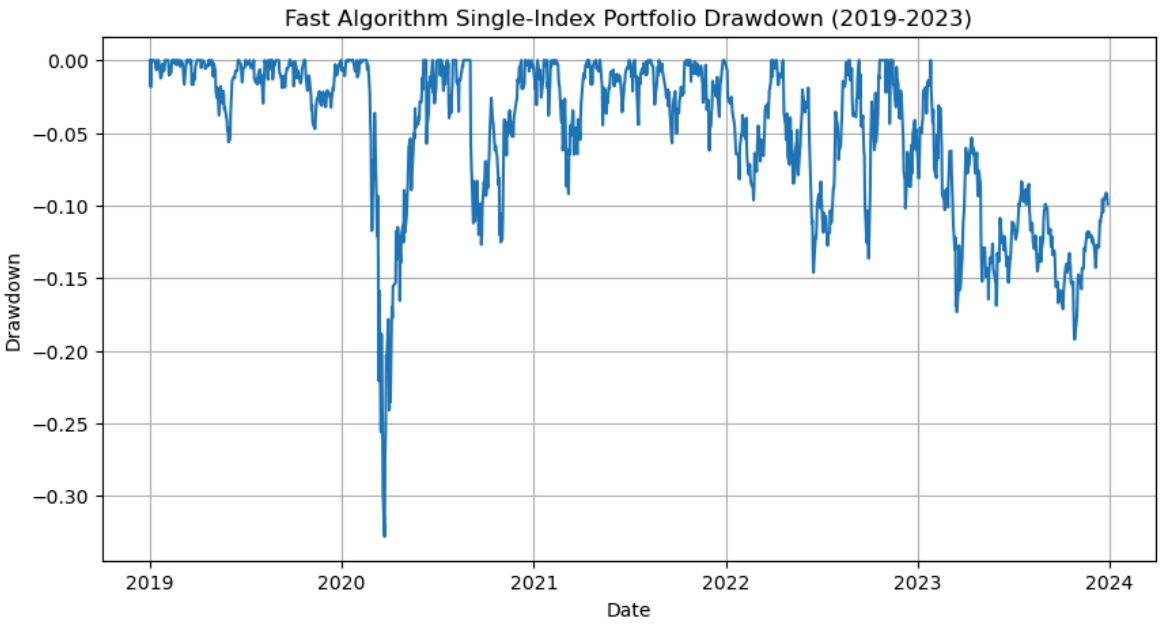
	Metric	Value
0	Backtest Start	2019-01-02
1	Backtest End	2023-12-29
2	Trading Days	1258
3	Cumulative Return	1.217178
4	Annualized Return	0.172925
5	Annualized Volatility	0.235196
6	Risk-Free Annual Return	0.017241
7	Sharpe Ratio	0.723554
8	Sortino Ratio	0.847689
9	Maximum Drawdown	-0.328097
10	Daily VaR 95%	-0.022058
11	Daily CVaR 95%	-0.034752
12	Beta vs SPY	0.916333
13	Jensen Alpha Annualized	0.032539
14	Tracking Error vs SPY	0.136629
15	Information Ratio vs SPY	0.146178

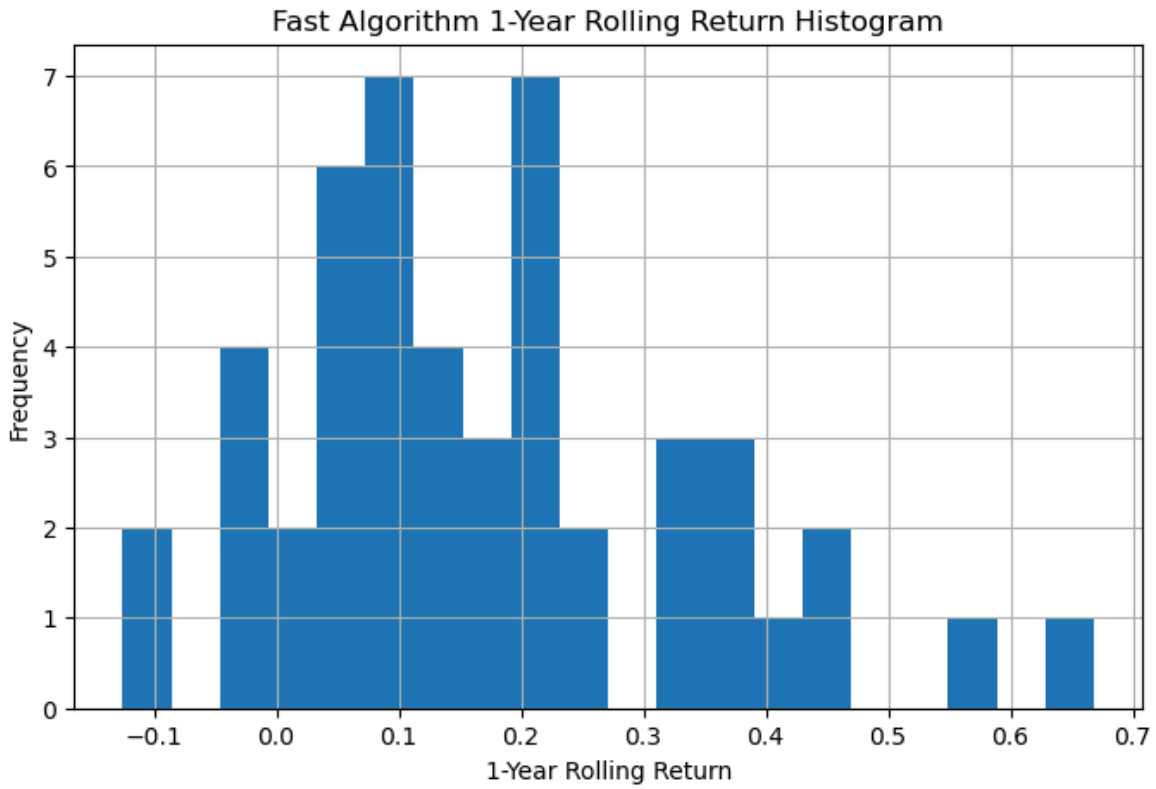
Rebalance Weights:

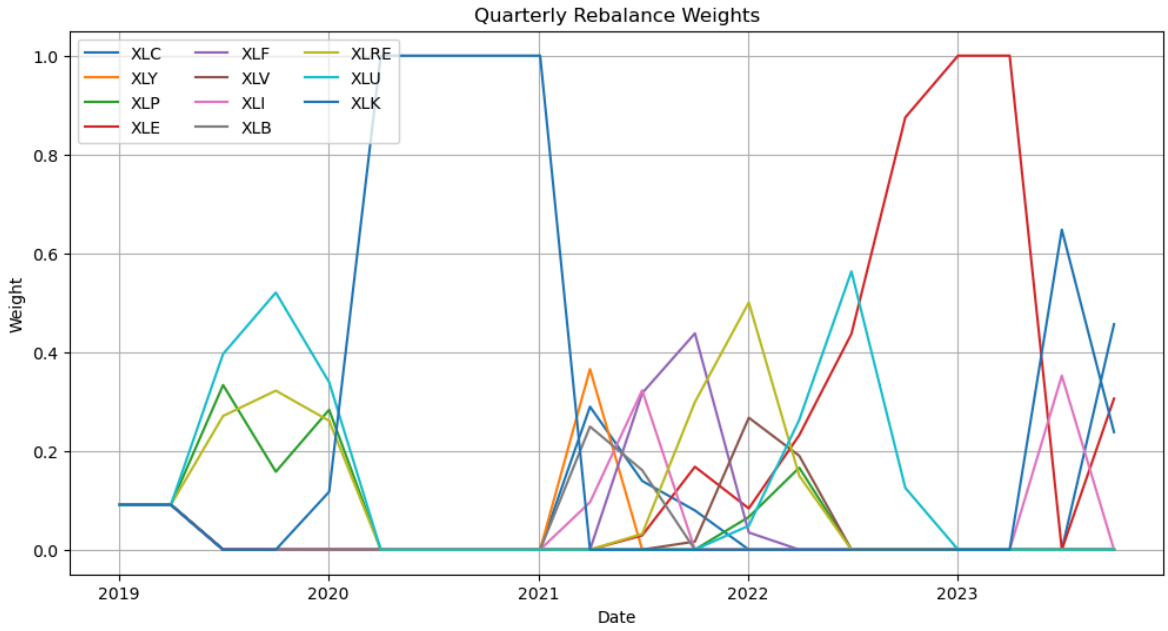
	Date	XLC	XLY	XLP	XLE	XLF	XLV	XLI	XLB	XLRE	XLU	XLK
0	2019-01-02	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909
1	2019-04-01	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909
2	2019-07-01	0.000000	0.000000	0.333209	0.000000	0.000000	0.000000	0.000000	0.000000	0.270628	0.396162	0.000000
3	2019-10-01	0.000000	0.000000	0.157712	0.000000	0.000000	0.000000	0.000000	0.000000	0.321863	0.520425	0.000000
4	2020-01-02	0.000000	0.000000	0.283099	0.000000	0.000000	0.000000	0.000000	0.000000	0.260832	0.338952	0.117116
5	2020-04-01	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000
6	2020-07-01	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000
7	2020-10-01	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000
8	2021-01-04	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000
9	2021-04-01	0.289368	0.365546	0.000000	0.000000	0.000000	0.000000	0.095952	0.249133	0.000000	0.000000	0.000000
10	2021-07-01	0.139090	0.000000	0.000000	0.028762	0.316769	0.000000	0.322558	0.160929	0.031892	0.000000	0.000000
11	2021-10-01	0.079295	0.000000	0.000000	0.167758	0.437830	0.016404	0.000000	0.000000	0.298713	0.000000	0.000000
12	2022-01-03	0.000000	0.000000	0.066112	0.083429	0.034680	0.267025	0.000000	0.000000	0.500320	0.048433	0.000000
13	2022-04-01	0.000000	0.000000	0.165853	0.231720	0.000000	0.190004	0.000000	0.000000	0.149546	0.262877	0.000000
14	2022-07-01	0.000000	0.000000	0.000000	0.436801	0.000000	0.000000	0.000000	0.000000	0.000000	0.563199	0.000000
15	2022-10-03	0.000000	0.000000	0.000000	0.874984	0.000000	0.000000	0.000000	0.000000	0.000000	0.125016	0.000000
16	2023-01-03	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
17	2023-04-03	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
18	2023-07-03	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
19	2023-10-02	0.456319	0.000000	0.000000	0.305741	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

```
18  0.352190  0.000000  0.000000  0.000000  0.647810
19  0.000000  0.000000  0.000000  0.000000  0.237940
```









In [16]:
```
##Task 4（Custom Strategy: Risk adjust Momentum）
```

In [18]:
```
text = """The proposed custom strategy is a cross-sectional, risk-adjusted momen

At each monthly rebalance date, a 12-1 momentum signal is computed for each sect

This raw momentum is further adjusted by recent volatility to obtain a more stab

All sectors are then ranked based on this score, and only those with positive mo

Among them, the top-performing sectors are selected, and portfolio weights are a

The portfolio is fully invested, long-only, and rebalanced monthly.

All signals are constructed using only historical information available at the t
```

```
This strategy aims to capture persistent sector trends while controlling for vol
```

In [19]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv(
    r"D:\QP_project\Data File.csv",
    parse_dates=["Date"]
)

df.columns = df.columns.str.replace("\\", "", regex=False).str.strip()
df = df.sort_values("Date").reset_index(drop=True)

sector_cols = ["XLC", "XLY", "XLP", "XLE", "XLF", "XLV", "XLI", "XLB", "XLRE", "
use_cols = ["Date", "SPY", "BIL"] + sector_cols
data = df[use_cols].copy().sort_values("Date").reset_index(drop=True)

for col in ["SPY", "BIL"] + sector_cols:
    data[col + "_ret"] = data[col].pct_change()

data = data[(data["Date"] >= "2018-01-01") & (data["Date"] <= "2023-12-31")].cop

ret_cols = [c + "_ret" for c in sector_cols]

lookback_mom = 252
skip_recent = 21
vol_window = 60
top_n = 3

bt = data[data["Date"] >= "2019-01-01"].copy().reset_index(drop=True)
bt["month"] = bt["Date"].dt.to_period("M")
rebalance_dates = bt.groupby("month")["Date"].min().tolist()

weights_records = []
portfolio_returns = []

current_weights = pd.Series(0.0, index=sector_cols)

for i in range(len(bt)):
    current_date = bt.loc[i, "Date"]

    if current_date in rebalance_dates:
        est = data[data["Date"] < current_date].copy().reset_index(drop=True)
        w = pd.Series(0.0, index=sector_cols)

        scores = {}
        for col in sector_cols:
            s = est[col].dropna()
            if len(s) < lookback_mom + skip_recent:
                scores[col] = np.nan
                continue

            p_t1 = s.iloc[-(skip_recent + 1)]
            p_t252 = s.iloc[-(lookback_mom + skip_recent)]
            mom_12_1 = p_t1 / p_t252 - 1.0

            r = est[col + "_ret"].dropna()
            if len(r) < vol_window:
```

```
                scores[col] = np.nan
                continue

            vol = r.iloc[-vol_window:].std(ddof=1)
            if pd.isna(vol) or vol <= 1e-12:
                scores[col] = np.nan
                continue

            scores[col] = mom_12_1 / vol

        score_s = pd.Series(scores).replace([np.inf, -np.inf], np.nan).dropna()
        score_s = score_s[score_s > 0].sort_values(ascending=False)

        if len(score_s) > 0:
            selected = score_s.iloc[:top_n]
            w = selected / selected.sum()
            w = w.reindex(sector_cols).fillna(0.0)
        else:
            w = pd.Series(1.0 / len(sector_cols), index=sector_cols)

        current_weights = w.copy()

        rec = {"Date": current_date}
        for c in sector_cols:
            rec[c] = current_weights[c]
        weights_records.append(rec)

    day_ret = bt.loc[i, ret_cols].copy()
    day_ret.index = sector_cols

    valid = day_ret.notna() & (current_weights > 0)
    if valid.sum() == 0:
        port_ret = 0.0
    else:
        w = current_weights[valid]
        w = w / w.sum()
        port_ret = np.dot(w.values, day_ret[valid].astype(float).values)

    portfolio_returns.append(port_ret)

bt["port_ret"] = portfolio_returns
bt["rf_ret"] = bt["BIL_ret"]
bt["spy_ret"] = bt["SPY_ret"]

bt = bt.dropna(subset=["port_ret", "rf_ret", "spy_ret"]).reset_index(drop=True)

bt["nav"] = (1 + bt["port_ret"]).cumprod()
bt["cum_return"] = bt["nav"] - 1
bt["peak"] = bt["nav"].cummax()
bt["drawdown"] = bt["nav"] / bt["peak"] - 1

ann_factor = 252
days = len(bt)

ann_return = bt["nav"].iloc[-1] ** (ann_factor / days) - 1
ann_vol = bt["port_ret"].std(ddof=1) * np.sqrt(ann_factor)

excess_ret = bt["port_ret"] - bt["rf_ret"]
rf_ann = (1 + bt["rf_ret"].mean()) ** ann_factor - 1
sharpe = excess_ret.mean() / bt["port_ret"].std(ddof=1) * np.sqrt(ann_factor)
```

```
downside = bt.loc[bt["port_ret"] < 0, "port_ret"]
downside_dev = downside.std(ddof=1) * np.sqrt(ann_factor)
sortino = (ann_return - rf_ann) / downside_dev if downside_dev != 0 else np.nan

mdd = bt["drawdown"].min()

var_95 = bt["port_ret"].quantile(0.05)
cvar_95 = bt.loc[bt["port_ret"] <= var_95, "port_ret"].mean()

market_excess = bt["spy_ret"] - bt["rf_ret"]
portfolio_excess = bt["port_ret"] - bt["rf_ret"]

beta = portfolio_excess.cov(market_excess) / market_excess.var(ddof=1)
alpha_daily = portfolio_excess.mean() - beta * market_excess.mean()
alpha_ann = alpha_daily * ann_factor

active_ret = bt["port_ret"] - bt["spy_ret"]
tracking_error = active_ret.std(ddof=1) * np.sqrt(ann_factor)
information_ratio = active_ret.mean() / active_ret.std(ddof=1) * np.sqrt(ann_fac

rolling_1y = bt.set_index("Date")["nav"].pct_change(252)
rolling_1y_monthly = rolling_1y.resample("ME").last().dropna()

summary = pd.DataFrame({
    "Metric": [
        "Backtest Start",
        "Backtest End",
        "Trading Days",
        "Cumulative Return",
        "Annualized Return",
        "Annualized Volatility",
        "Risk-Free Annual Return",
        "Sharpe Ratio",
        "Sortino Ratio",
        "Maximum Drawdown",
        "Daily VaR 95%",
        "Daily CVaR 95%",
        "Beta vs SPY",
        "Jensen Alpha Annualized",
        "Tracking Error vs SPY",
        "Information Ratio vs SPY"
    ],
    "Value": [
        bt["Date"].min().strftime("%Y-%m-%d"),
        bt["Date"].max().strftime("%Y-%m-%d"),
        len(bt),
        bt["cum_return"].iloc[-1],
        ann_return,
        ann_vol,
        rf_ann,
        sharpe,
        sortino,
        mdd,
        var_95,
        cvar_95,
        beta,
        alpha_ann,
        tracking_error,
        information_ratio
```

```
    ]
})

weights_df = pd.DataFrame(weights_records)

print(summary)
print("\nMonthly Rebalance Weights:")
print(weights_df)

plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["nav"])
plt.title("Risk-Adjusted Momentum Sector Rotation NAV (2019-2023)")
plt.xlabel("Date")
plt.ylabel("NAV")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(bt["Date"], bt["drawdown"])
plt.title("Risk-Adjusted Momentum Sector Rotation Drawdown (2019-2023)")
plt.xlabel("Date")
plt.ylabel("Drawdown")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.hist(rolling_1y_monthly.values, bins=20)
plt.title("Momentum Strategy 1-Year Rolling Return Histogram")
plt.xlabel("1-Year Rolling Return")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

plt.figure(figsize=(12, 6))
for col in sector_cols:
    plt.plot(weights_df["Date"], weights_df[col], label=col)
plt.title("Monthly Rebalance Weights")
plt.xlabel("Date")
plt.ylabel("Weight")
plt.legend(ncol=3)
plt.grid(True)
plt.show()
```

	Metric	Value
0	Backtest Start	2019-01-02
1	Backtest End	2023-12-29
2	Trading Days	1258
3	Cumulative Return	1.288149
4	Annualized Return	0.180351
5	Annualized Volatility	0.223469
6	Risk-Free Annual Return	0.017241
7	Sharpe Ratio	0.777669
8	Sortino Ratio	0.924072
9	Maximum Drawdown	-0.303232
10	Daily VaR 95%	-0.020134
11	Daily CVaR 95%	-0.032829
12	Beta vs SPY	0.876558
13	Jensen Alpha Annualized	0.042122
14	Tracking Error vs SPY	0.129573
15	Information Ratio vs SPY	0.181984

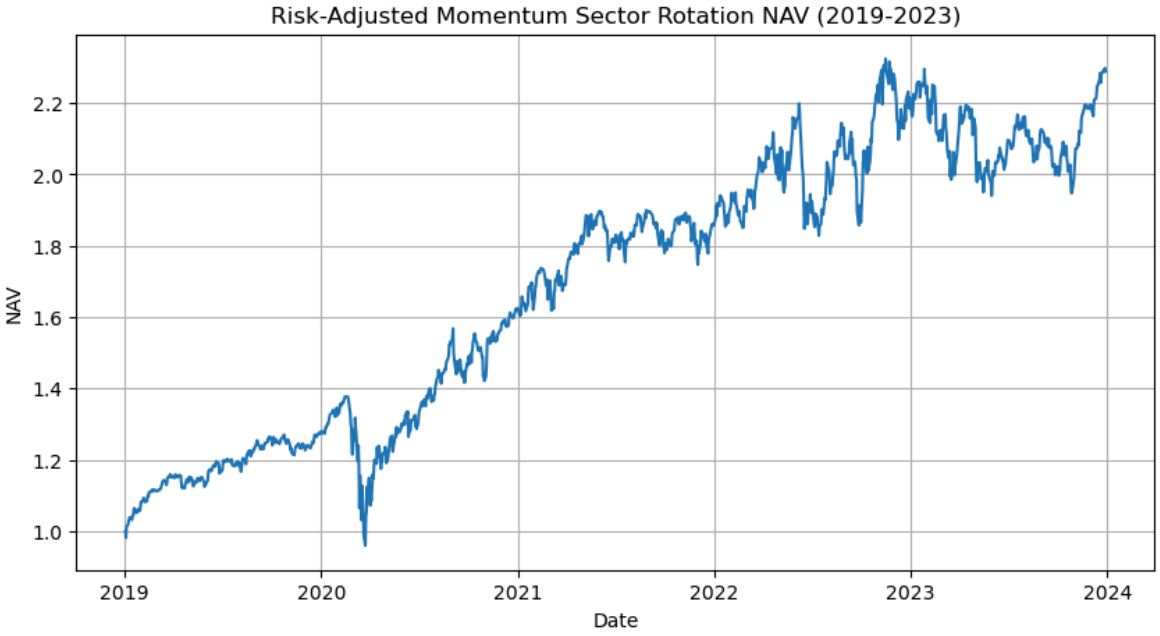
Monthly Rebalance Weights:

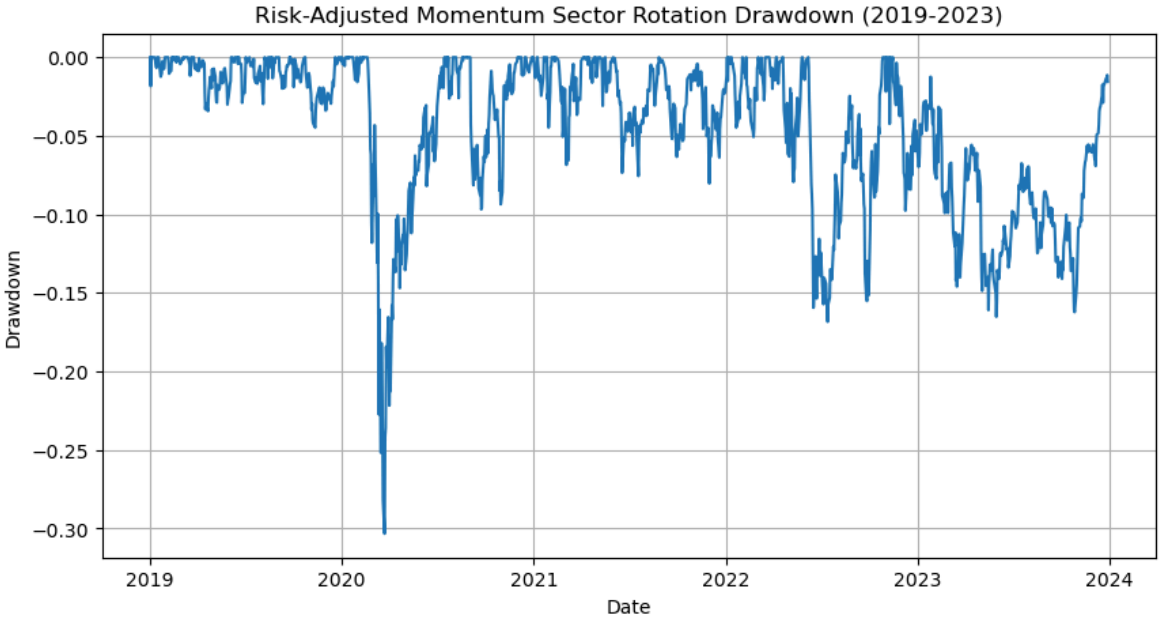
	Date	XLC	XLY	XLP	XLE	XLF	XLV	\
0	2019-01-02	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	
1	2019-02-01	0.090909	0.090909	0.090909	0.090909	0.090909	0.090909	
2	2019-03-01	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
3	2019-04-01	0.000000	0.000000	0.000000	0.000000	0.000000	0.162301	
4	2019-05-01	0.000000	0.250331	0.000000	0.000000	0.000000	0.000000	
5	2019-06-03	0.000000	0.000000	0.359537	0.000000	0.000000	0.000000	
6	2019-07-01	0.000000	0.000000	0.347319	0.000000	0.000000	0.000000	
7	2019-08-01	0.000000	0.000000	0.353480	0.000000	0.000000	0.000000	
8	2019-09-03	0.000000	0.000000	0.278303	0.000000	0.000000	0.000000	
9	2019-10-01	0.000000	0.000000	0.262151	0.000000	0.000000	0.000000	
10	2019-11-01	0.000000	0.000000	0.216587	0.000000	0.000000	0.000000	
11	2019-12-02	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
12	2020-01-02	0.000000	0.000000	0.314754	0.000000	0.000000	0.000000	
13	2020-02-03	0.000000	0.000000	0.373536	0.000000	0.000000	0.000000	
14	2020-03-02	0.000000	0.000000	0.301203	0.000000	0.000000	0.000000	
15	2020-04-01	0.000000	0.000000	0.287965	0.000000	0.000000	0.000000	
16	2020-05-01	0.000000	0.000000	0.132081	0.000000	0.000000	0.000000	
17	2020-06-01	0.000000	0.000000	0.183506	0.000000	0.000000	0.419866	
18	2020-07-01	0.271758	0.000000	0.000000	0.000000	0.000000	0.272150	
19	2020-08-03	0.211963	0.000000	0.000000	0.000000	0.000000	0.208948	
20	2020-09-01	0.000000	0.242184	0.000000	0.000000	0.000000	0.278900	
21	2020-10-01	0.000000	0.303710	0.000000	0.000000	0.000000	0.286860	
22	2020-11-02	0.000000	0.316277	0.000000	0.000000	0.000000	0.279474	
23	2020-12-01	0.254974	0.337666	0.000000	0.000000	0.000000	0.000000	
24	2021-01-04	0.256436	0.353737	0.000000	0.000000	0.000000	0.000000	
25	2021-02-01	0.261747	0.320788	0.000000	0.000000	0.000000	0.000000	
26	2021-03-01	0.264776	0.351694	0.000000	0.000000	0.000000	0.000000	
27	2021-04-01	0.360238	0.000000	0.000000	0.000000	0.000000	0.000000	
28	2021-05-03	0.000000	0.000000	0.000000	0.000000	0.301472	0.000000	
29	2021-06-01	0.000000	0.000000	0.000000	0.000000	0.329901	0.000000	
30	2021-07-01	0.000000	0.000000	0.000000	0.000000	0.331842	0.000000	
31	2021-08-02	0.327063	0.000000	0.000000	0.000000	0.334607	0.000000	
32	2021-09-01	0.361573	0.000000	0.000000	0.000000	0.000000	0.000000	
33	2021-10-01	0.317577	0.000000	0.000000	0.000000	0.355237	0.000000	
34	2021-11-01	0.264031	0.000000	0.000000	0.352052	0.383917	0.000000	
35	2021-12-01	0.000000	0.000000	0.000000	0.359249	0.367758	0.000000	
36	2022-01-03	0.000000	0.000000	0.000000	0.333075	0.315605	0.000000	
37	2022-02-01	0.000000	0.000000	0.000000	0.301434	0.000000	0.322581	
38	2022-03-01	0.000000	0.000000	0.235734	0.490248	0.274018	0.000000	
39	2022-04-01	0.000000	0.000000	0.318483	0.375795	0.000000	0.000000	

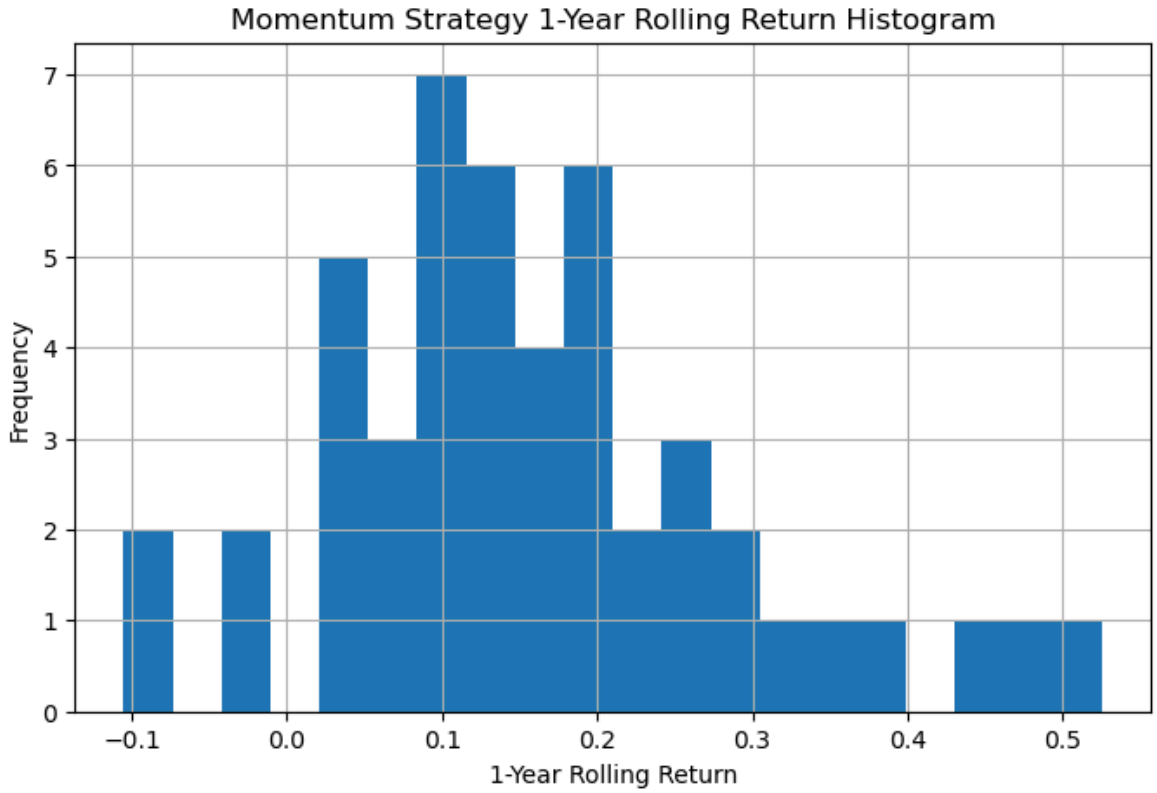
40	2022-05-02	0.000000	0.000000	0.000000	0.468409	0.000000	0.000000
41	2022-06-01	0.000000	0.000000	0.223274	0.553718	0.000000	0.000000
42	2022-07-01	0.000000	0.000000	0.000000	0.542645	0.000000	0.164182
43	2022-08-01	0.000000	0.000000	0.147897	0.534321	0.000000	0.000000
44	2022-09-01	0.000000	0.000000	0.184250	0.572639	0.000000	0.000000
45	2022-10-03	0.000000	0.000000	0.071740	0.757364	0.000000	0.000000
46	2022-11-01	0.000000	0.000000	0.000000	0.841620	0.000000	0.000000
47	2022-12-01	0.000000	0.000000	0.119157	0.839745	0.000000	0.000000
48	2023-01-03	0.000000	0.000000	0.209878	0.657013	0.000000	0.133110
49	2023-02-01	0.000000	0.000000	0.010234	0.930703	0.000000	0.000000
50	2023-03-01	0.000000	0.000000	0.000000	0.674915	0.000000	0.181500
51	2023-04-03	0.000000	0.000000	0.000000	0.713081	0.000000	0.000000
52	2023-05-01	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000
53	2023-06-01	0.000000	0.000000	0.000000	0.426059	0.000000	0.000000
54	2023-07-03	0.160882	0.000000	0.000000	0.000000	0.000000	0.000000
55	2023-08-01	0.000000	0.255219	0.000000	0.000000	0.000000	0.000000
56	2023-09-01	0.314722	0.000000	0.000000	0.000000	0.000000	0.000000
57	2023-10-02	0.328574	0.000000	0.000000	0.000000	0.000000	0.000000
58	2023-11-01	0.340665	0.000000	0.000000	0.000000	0.000000	0.000000
59	2023-12-01	0.475090	0.000000	0.000000	0.000000	0.000000	0.000000

	XLI	XLB	XLRE	XLU	XLK
0	0.090909	0.090909	0.090909	0.090909	0.090909
1	0.090909	0.090909	0.090909	0.090909	0.090909
2	0.000000	0.000000	0.484914	0.515086	0.000000
3	0.000000	0.000000	0.444149	0.393550	0.000000
4	0.000000	0.000000	0.348852	0.400816	0.000000
5	0.000000	0.000000	0.331362	0.309101	0.000000
6	0.000000	0.000000	0.321256	0.331424	0.000000
7	0.000000	0.000000	0.296552	0.349968	0.000000
8	0.000000	0.000000	0.314669	0.407028	0.000000
9	0.000000	0.000000	0.326513	0.411335	0.000000
10	0.000000	0.000000	0.371276	0.412137	0.000000
11	0.000000	0.000000	0.355529	0.342752	0.301720
12	0.000000	0.000000	0.000000	0.286896	0.398350
13	0.000000	0.000000	0.000000	0.283770	0.342694
14	0.000000	0.000000	0.000000	0.338736	0.360061
15	0.000000	0.000000	0.000000	0.258365	0.453670
16	0.000000	0.000000	0.000000	0.000000	0.867919
17	0.000000	0.000000	0.000000	0.000000	0.396628
18	0.000000	0.000000	0.000000	0.000000	0.456092
19	0.000000	0.000000	0.000000	0.000000	0.579089
20	0.000000	0.000000	0.000000	0.000000	0.478916
21	0.000000	0.000000	0.000000	0.000000	0.409430
22	0.000000	0.000000	0.000000	0.000000	0.404249
23	0.000000	0.000000	0.000000	0.000000	0.407361
24	0.000000	0.000000	0.000000	0.000000	0.389827
25	0.000000	0.000000	0.000000	0.000000	0.417465
26	0.000000	0.000000	0.000000	0.000000	0.383530
27	0.000000	0.319215	0.000000	0.000000	0.320547
28	0.355524	0.343005	0.000000	0.000000	0.000000
29	0.347033	0.323066	0.000000	0.000000	0.000000
30	0.357552	0.310607	0.000000	0.000000	0.000000
31	0.338331	0.000000	0.000000	0.000000	0.000000
32	0.318348	0.000000	0.000000	0.000000	0.320080
33	0.000000	0.000000	0.327186	0.000000	0.000000
34	0.000000	0.000000	0.000000	0.000000	0.000000
35	0.000000	0.000000	0.272993	0.000000	0.000000
36	0.000000	0.000000	0.351320	0.000000	0.000000
37	0.000000	0.000000	0.375985	0.000000	0.000000

38	0.000000	0.000000	0.000000	0.000000	0.000000
39	0.000000	0.000000	0.305722	0.000000	0.000000
40	0.000000	0.000000	0.262820	0.268771	0.000000
41	0.000000	0.000000	0.000000	0.223008	0.000000
42	0.000000	0.000000	0.000000	0.293173	0.000000
43	0.000000	0.000000	0.000000	0.317782	0.000000
44	0.000000	0.000000	0.000000	0.243112	0.000000
45	0.000000	0.000000	0.000000	0.170896	0.000000
46	0.000000	0.000000	0.000000	0.158380	0.000000
47	0.000000	0.000000	0.000000	0.041098	0.000000
48	0.000000	0.000000	0.000000	0.000000	0.000000
49	0.000000	0.000000	0.000000	0.059063	0.000000
50	0.000000	0.000000	0.000000	0.143585	0.000000
51	0.184718	0.102200	0.000000	0.000000	0.000000
52	0.000000	0.000000	0.000000	0.000000	0.000000
53	0.305476	0.000000	0.000000	0.000000	0.268466
54	0.184516	0.000000	0.000000	0.000000	0.654602
55	0.362200	0.000000	0.000000	0.000000	0.382581
56	0.373964	0.000000	0.000000	0.000000	0.311314
57	0.327871	0.000000	0.000000	0.000000	0.343555
58	0.282972	0.000000	0.000000	0.000000	0.376363
59	0.104739	0.000000	0.000000	0.000000	0.420171






Momentum Strategy 1-Year Rolling Return Histogram
Frequency
1-Year Rolling Return
0
1
2
3
4
5
6
7
−0.1
0.0
0.1
0.2
0.3
0.4
0.5


Monthly Rebalance Weights
XLC
XLY
XLP
XLE
XLF
XLV
XLI
XLB
XLRE
XLU
XLK
Weight
Date
0.0
0.2
0.4
0.6
0.8
1.0
2019
2020
2021
2022
2023
2024